

CinéSearch

Moteur de Recherche de Films avec Elasticsearch

Théo Farris

Camille Laborde

Guillaume Savoye

Sarra El-Faiez

Rapport de projet

Année universitaire 2025-2026 — CY-Tech Cergy

0. Plan

1. Introduction	3
2. Arborescence du Projet	3
3. Fichiers Livrés	4
4. Détail par Partie	4
Partie 1 — Infrastructure (15 pts)	4
Partie 2 — Indexation (15 pts)	4
Partie 3 — Recherche (30 pts)	5
Partie 4 — Agrégations (20 pts)	5
Partie 5 — Interface Streamlit (10 pts)	5
Partie 6 — Kibana (10 pts)	6
5. Lancement de l'Application Ciné Search	9
Prérequis	9
Lancement sur Windows	9
Lancement sur Linux / macOS	10
Accès aux interfaces	10
Arrêt de l'application	10
6. Conclusion	11
7. Annexe — Captures d'Écran de l'Interface CinéSearch	11

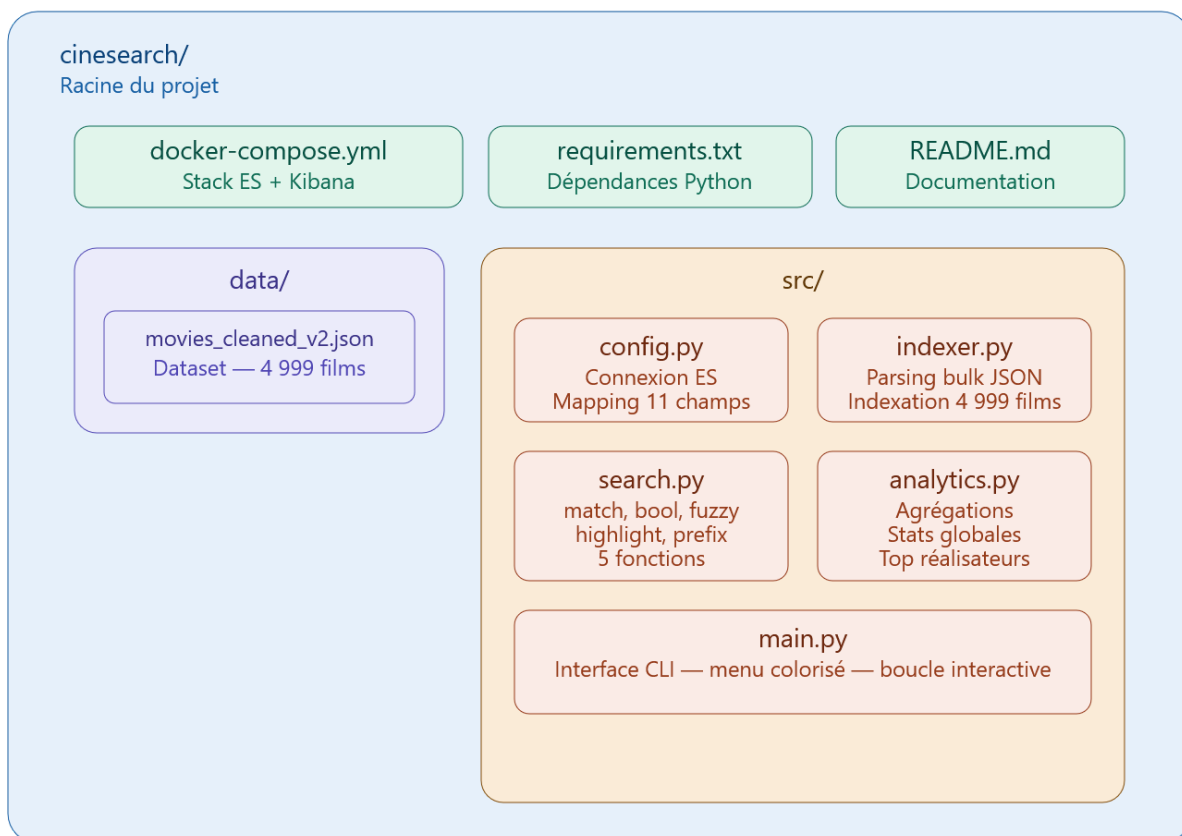
1. Introduction

Ce projet consiste à concevoir et développer **CinéSearch**, un moteur de recherche de films s'appuyant sur Elasticsearch comme backend et Python comme langage de développement. À partir d'un dataset de près de 5 000 films, il offre une interface en ligne de commande permettant d'effectuer des recherches variées : par titre, par synopsis, par critères avancés (acteur, réalisateur, genre, note, année), avec tolérance aux fautes de frappe et auto-complétion.

L'infrastructure est entièrement conteneurisée via Docker Compose, garantissant la portabilité et la reproductibilité de l'environnement. Les données sont indexées dans Elasticsearch en tirant parti de l'API bulk pour des performances optimales, et l'interface exploite le Query DSL pour construire des requêtes précises et pertinentes. Des agrégations permettent d'extraire des statistiques avancées sur le catalogue, tandis qu'un dashboard Kibana offre une visualisation interactive des données.

2. Arborescence du Projet

Voici la structure finale de notre projet :



3. Fichiers Livrés

Voici l'ensemble des fichiers qui ont été produits, testés et sont fonctionnels :

Fichier	Partie	Description
docker-compose.yml	Partie 1	Stack ES 8.13 + Kibana avec réseau nommé, healthcheck, volume persistant
src/config.py	Partie 1	Mapping explicite des 11 champs, connexion ES, création d'index
src/indexer.py	Partie 2	Conversion encodage, parsing bulk, indexation 4999 films, vérification
src/search.py	Partie 3	5 fonctions : <code>search_by_title</code> , <code>search_advanced</code> , <code>search_plot</code> , <code>search_fuzzy</code> , <code>suggest_titles</code>
src/analytics.py	Partie 4	Aggregation et statistiques : <code>global_stats</code> , <code>top_genres</code> , <code>top_directors</code> , <code>top_actors</code> , <code>film_by_decade</code> , <code>avg_rating_by_year</code> , <code>best_rated_genres</code> , <code>best_rated_directors</code>
src/main.py	Partie 5	Interface CLI avec menu colorisé (<code>colorama</code>), boucle interactive, gestion des erreurs

4. Détail par Partie

Partie 1 — Infrastructure (15 pts)

Le fichier ***“docker-compose.yml”*** déploie Elasticsearch 8.13.0 et Kibana 8.13.0 dans des conteneurs Docker séparés reliés par un réseau nommé `elastic`. Elasticsearch démarre en nœud unique avec la sécurité désactivée (mode développement). Un healthcheck garantit que Kibana ne démarre qu’une fois Elasticsearch réellement prêt. Les données sont persistées via un volume nommé `es-data`.

Le fichier ***“config.py”*** crée programmatiquement l’index `movies` avec un mapping explicite de 11 champs, conforme au tableau du cours (Chapitre 5.7). Les champs `title`, `directors` et `actors` utilisent des multi-fields (`text` + `keyword`) pour permettre à la fois la recherche full-text et les agrégations exactes. Le champ `image_url` est marqué *index: false* car il n’a pas vocation à être recherché.

Partie 2 — Indexation (15 pts)

Le fichier ***“indexer.py”*** charge le dataset `movies_cleaned_v2.json` en gérant la conversion d’encodage latin-1 vers UTF-8. Le fichier est au format bulk Elasticsearch (lignes alternées méta/document) : le script parse ces lignes par paires, extrait l’ID via `meta["_id"]` et les données via `doc["fields"]`.

L’indexation utilise `helpers.bulk()` avec `stats_only=True` pour des performances optimales. Résultat obtenu : 4999 films indexés avec succès, 0 erreur. Une vérification post-indexation via `es.count()` et `es.search(size=3)` confirme l’intégrité des données.

Partie 3 — Recherche (30 pts)

Le fichier ***“search.py”*** implémente les 5 fonctions de recherche demandées par l'énoncé :

Fonction	Type de requête	Description
search_by_title()	Match query	Recherche full-text sur le titre.
search_advanced()	Bool query	Multi-critères dynamiques : titre, acteur, réalisateur, genre, note, année.
search_plot()	Match + Highlight	Recherche dans le synopsis avec surlignage des termes trouvés.
search_fuzzy()	Fuzzy query	Tolérance aux fautes de frappe. Ex: “Inception” → “Inception”.
suggest_titles()	Prefix query	Auto-complétion sur title.keyword. Option 1 de l'énoncé.

La fonction *search_advanced()* est la plus complète : elle construit dynamiquement une bool query en n'ajoutant chaque critère que s'il est fourni. Les critères full-text (titre, acteur, réalisateur) sont placés en clause must (contribuent au score de pertinence) et les filtres exacts (genre, plage de notes, plage d'années) en clause filter (plus performants, mis en cache par ES).

Partie 4 — Agrégations (20 pts)

Le fichier ***“analytics.py”*** implémente l'ensemble des fonctions d'agrégation demandées, exploitant les agrégations Elasticsearch pour produire des statistiques détaillées sur le dataset de films.

La fonction *global_stats()* calcule les statistiques globales du dataset : nombre total de films, note moyenne, minimale et maximale, quartiles (Q1, Q2, Q3) via une agrégation *percentiles*, ainsi que le meilleur et le pire film récupérés via des agrégations *top_hits*.

Les analyses par catégorie sont réparties en quatre fonctions : *top_genres()* et *top_directors()* utilisent des *terms aggregations* sur les champs *genres* et *directors.keyword* pour identifier les genres et réalisateurs les plus représentés. *top_actors()* applique le même principe sur *actors.keyword*. *films_by_decade()* utilise une *histogram aggregation* sur le champ *year* avec un intervalle de 10 ans, affichant une barre proportionnelle colorisée pour chaque décennie.

Les analyses avancées reposent sur des agrégations imbriquées : *avg_rating_by_year()* combine une *terms aggregation* sur *year* avec une sous-agrégation *avg* sur *rating* pour suivre l'évolution de la note moyenne par année. *bestRatedGenres()* trie les genres par note moyenne décroissante. Enfin, *bestRatedDirectors()* utilise un *bucket_selector* pour filtrer les réalisateurs ayant réalisé au minimum 3 films, puis trie les résultats par note moyenne décroissante côté Python.

Partie 5 — Interface Streamlit (10 pts)

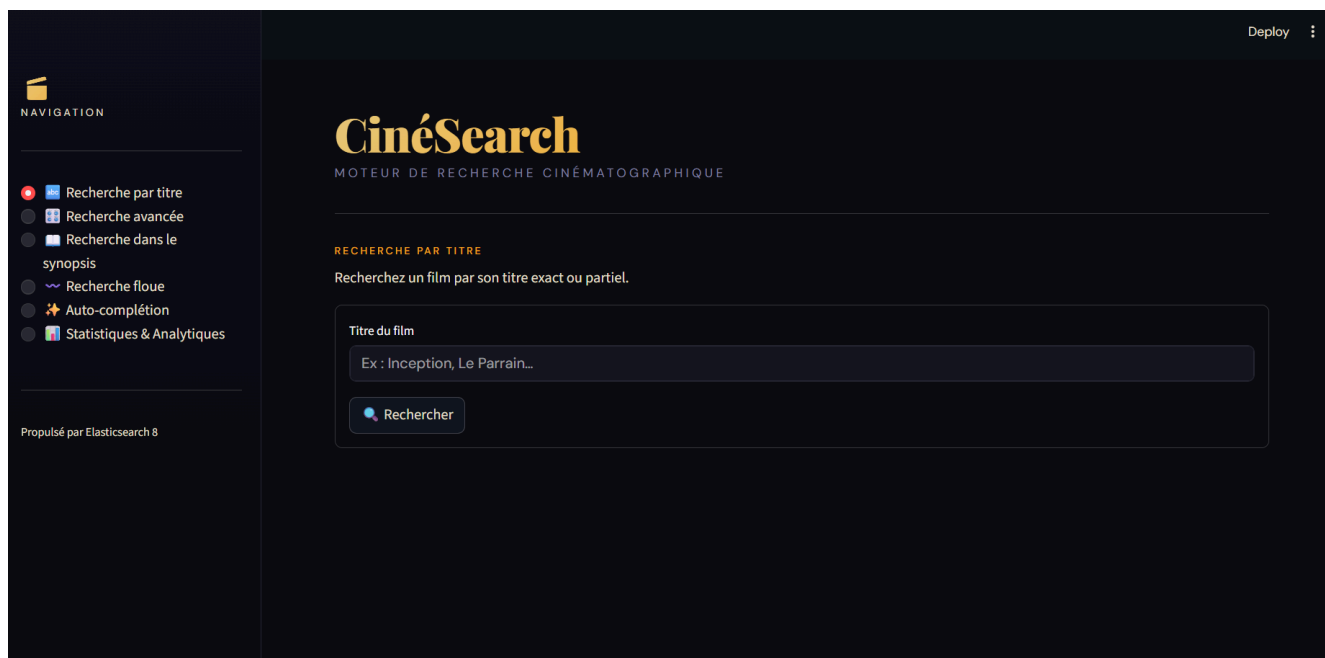
Le fichier ***“main.py”*** fournit une interface web interactive développée avec la bibliothèque Streamlit, accessible sur <http://localhost:8501>. L'application adopte un thème cinématographique sombre avec

des accents dorés, utilisant des polices Google Fonts (*Playfair Display* pour les titres, *DM Sans* pour le texte).

La navigation est assurée par une sidebar à gauche proposant 6 sections : Recherche par titre, Recherche avancée, Recherche dans le synopsis, Recherche floue, Auto-complétion, et Statistiques & Analytiques. Chaque section dispose d'un formulaire dédié avec validation des saisies (champs vides, caractères non autorisés, plages invalides) et affiche un message d'erreur explicite en cas de saisie incorrecte.

Les résultats sont présentés sous forme de cards HTML avec le titre en doré, la note, le genre, le réalisateur et le score de pertinence. La recherche dans le synopsis affiche en plus des fragments du texte avec les termes trouvés mis en évidence. La page Statistiques & Analytiques expose les données de *“analytics.py”* : statistiques globales (total, note moyenne, meilleur/pire film, quartiles), top genres/réalisateurs/acteurs sous forme de tableaux, distribution par décennie en barres horizontales, et analyses avancées (note moyenne par année, genres et réalisateurs les mieux notés).

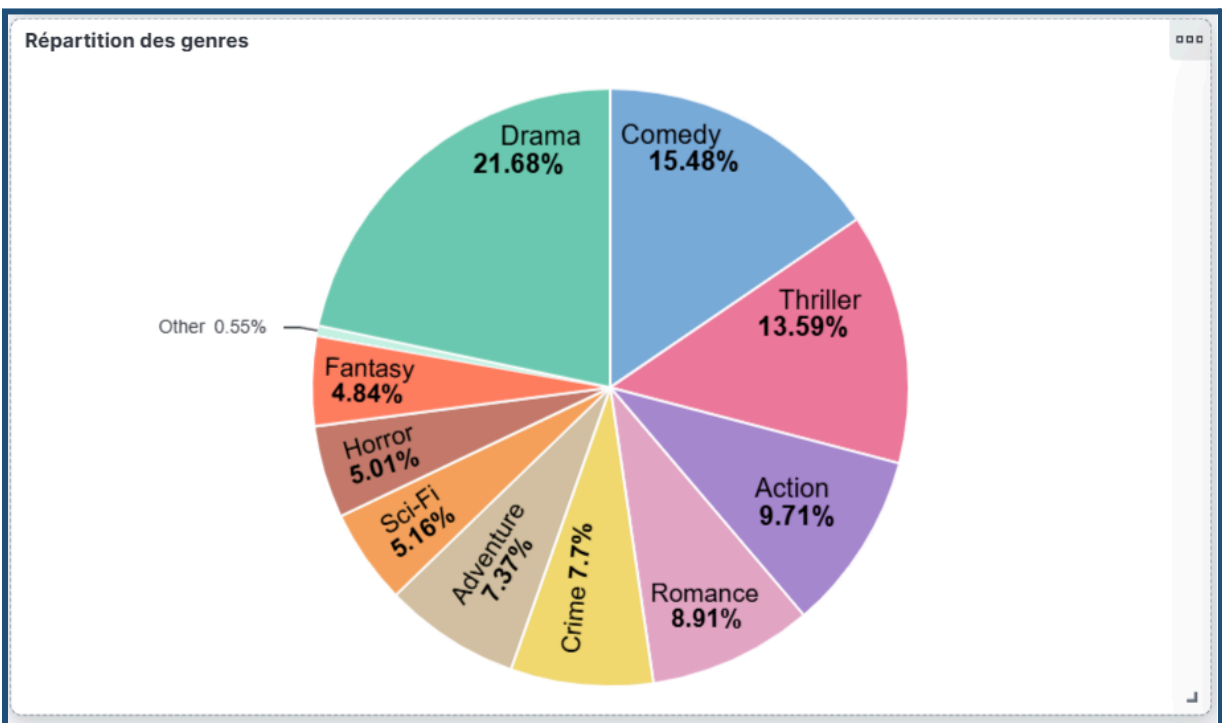
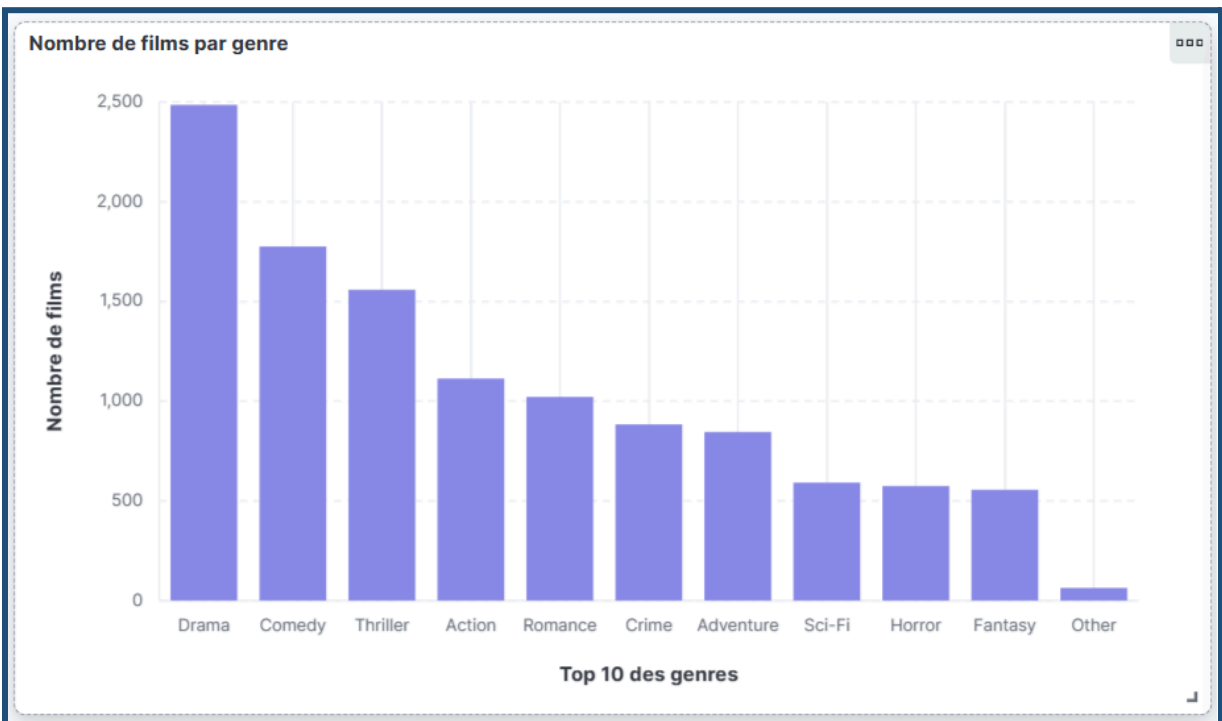
Figure 1 — Page d'accueil (Recherche par titre) : La page d'accueil présente le formulaire de recherche par titre. La sidebar de navigation à gauche liste les 6 sections disponibles, avec un indicateur de statut de connexion Elasticsearch en bas.



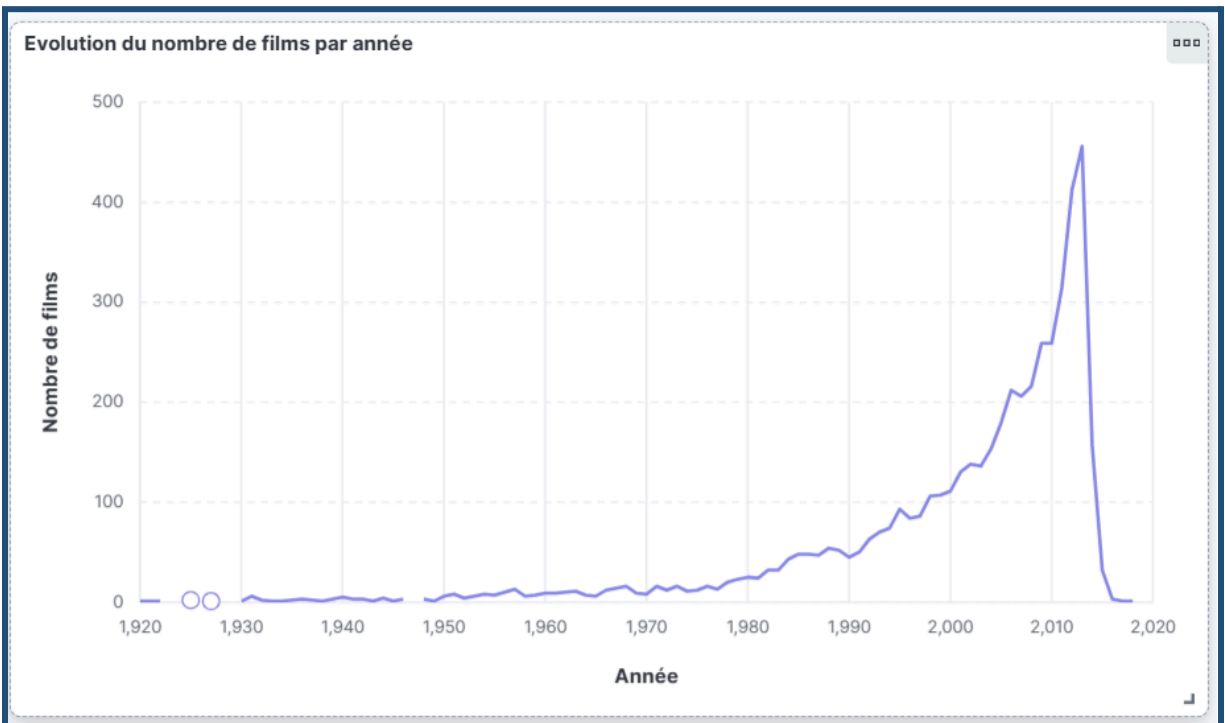
Partie 6 — Kibana (10 pts)

Le tableau de bord “CinéSearch — Tableau de bord” a été conçu pour offrir une vue d'ensemble interactive et intuitive de notre catalogue films, facilitant l'exploration des données indexées. Il intègre les visualisations suivantes :

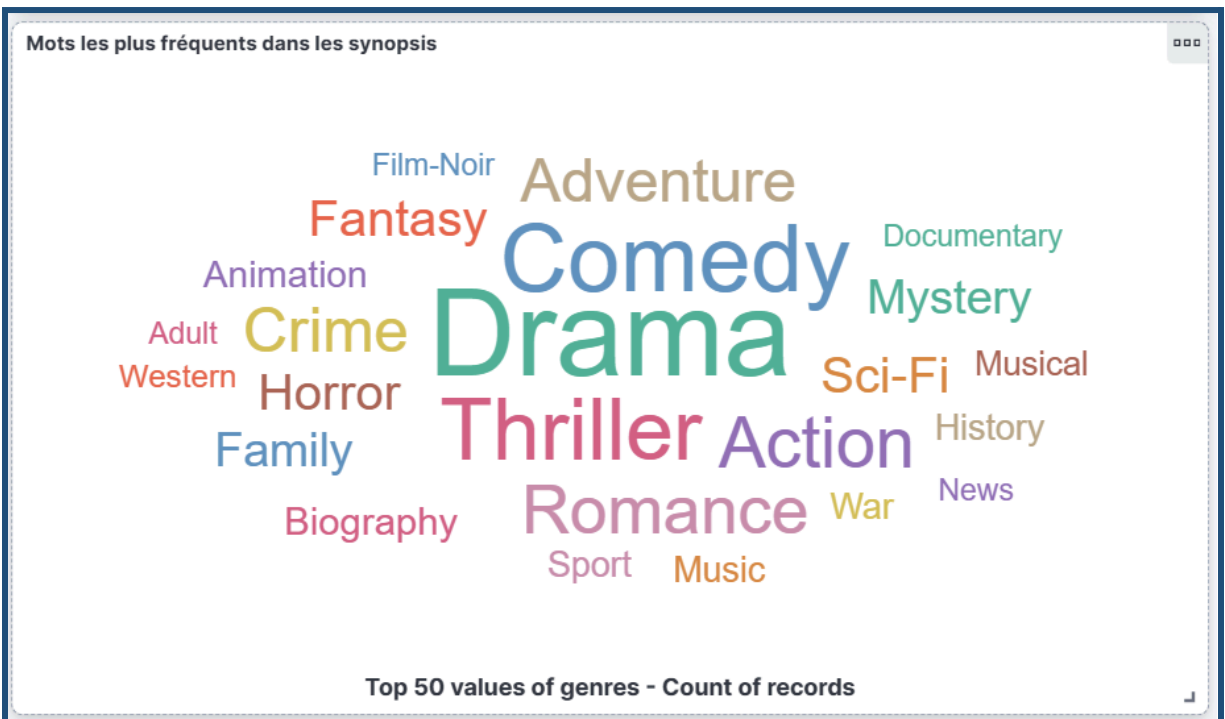
- **Répartition des genres** : Un *Pie chart* et un *Bar chart* permettent de visualiser immédiatement les genres dominants du dataset. Le genre “Drama” arrive en tête avec 21,68 % des titres, suivi par la “Comedy” (15,48 %) et le “Thriller” (13,59 %), offrant une vision claire de la composition du catalogue.



- **Évolution temporelle** : Un *Line chart* illustre l'évolution du nombre de films par année. Ce graphique met en évidence une croissance exponentielle de la production cinématographique représentée dans notre base, avec un pic significatif observé autour de la période 2010-2015. La forte chute après 2025 est sûrement dû à un problème de données, ne permettant pas d'analyser correctement les données sur cette période là.



- **Analyse textuelle** : Un *Tag cloud* a été généré à partir du champ *genre*, permettant de faire ressortir visuellement les genres les plus récurrents des films.



- **Top 10 des films** : Une *Data table* récapitulative affiche les 10 films les mieux notés du dataset, incluant le titre, l'année de sortie, la note IMDb et le réalisateur, offrant une référence rapide sur les œuvres les plus plébiscitées.

Top 10 des films les mieux notés			
Film	Rating	Année	Réalisateur
The Shawshank Redemp	9.3	1,994	Frank Darabont
The Godfather	9.2	1,972	Francis Ford Coppola
Il buono, il brutto, il cattiv	9	1,966	Sergio Leone
Pulp Fiction	9	1,994	Quentin Tarantino
The Dark Knight	9	2,008	Christopher Nolan
The Godfather: Part II	9	1,974	Francis Ford Coppola
12 Angry Men	8.9	1,957	Sidney Lumet
Schindler's List	8.9	1,993	Steven Spielberg
The Lord of the Rings: Tr	8.9	2,003	Peter Jackson
Fight Club	8.8	1,999	David Fincher

La création de ce tableau de bord a nécessité la mise en place préalable d'un *Data View* ciblant l'index *movies*. Cette étape permet de valider visuellement la qualité de notre mapping et la pertinence de nos données indexées, tout en offrant aux utilisateurs finaux un outil de pilotage performant pour analyser les tendances du cinéma mondial.

5. Lancement de l'Application Ciné Search

Notre application CinéSearch repose sur trois services Docker (Elasticsearch, Kibana et Streamlit) orchestrés via Docker Compose. Nous avons également créé un script de démarrage qui automatise le lancement et l'ouverture des interfaces dans le navigateur.

Prérequis

Avant de lancer l'application, il faut vous s'assurer que les éléments suivants sont installés sur votre machine :

- Docker Desktop (Windows / Mac) ou Docker Engine + Docker Compose (Linux).
- Git (optionnel, pour cloner le dépôt).

Lancement sur Windows

Depuis l'explorateur de fichiers, naviguer jusqu'au dossier du projet et double-cliquer sur le fichier **start.bat**.

Une fenêtre de terminal s'ouvre et exécute les étapes suivantes :

```
C:\WINDOWS\system32\cmd x + v

Demarrage de CineSearch
=====

[1/4] Demarrage des services Docker...
[+] up 4/4
✓Network cinesearch-streamlit_elastic Created 0.0ss
✓Container elasticsearch Healthy 16.0s
✓Container kibana Started 16.2s
✓Container cinesearch_ui Started 16.2s

[2/4] Attente d'Elasticsearch...
      OK - Elasticsearch est pret !
[3/4] Attente de Streamlit...
      OK - Streamlit est pret !
[4/4] Attente de Kibana...
      OK - Kibana est pret !

=====
Ouverture des pages dans le navigateur
=====

CineSearch : http://localhost:8501
Kibana      : http://localhost:5601

=====
Tout est lance ! Appuie sur une touche
pour fermer cette fenetre.
=====
```

- Démarrage des conteneurs Docker en arrière-plan.
- Attente qu'Elasticsearch, Streamlit et Kibana soient pleinement opérationnels.
- Ouverture automatique des deux interfaces dans le navigateur par défaut.

La fenêtre de terminal reste ouverte jusqu'à ce que l'utilisateur appuie sur une touche pour la fermer.

Lancement sur Linux / macOS

Ouvrir un terminal dans le dossier du projet et exécuter les commandes suivantes :

```
chmod +x start.sh      # Rendre le script exécutable (une seule fois)
./start.sh             # Lancer l'application
```

Le script effectue les mêmes opérations que sur Windows :

- Démarrage des conteneurs Docker en arrière-plan.
- Vérification de la disponibilité de chaque service.
- Ouverture automatique des interfaces dans le navigateur.

Accès aux interfaces

Une fois les services démarrés, les interfaces sont accessibles aux adresses suivantes :

- CinéSearch (interface de recherche) : **http://localhost:8501**
- Kibana (tableau de bord Elasticsearch) : **http://localhost:5601**

Arrêt de l'application

Pour arrêter tous les services, exécuter la commande suivante dans le dossier du projet :

```
docker compose down
```

6. Conclusion

Ce projet nous a permis de concevoir et de déployer un moteur de recherche de films complet, de l'infrastructure jusqu'à l'interface utilisateur. En s'appuyant sur Elasticsearch et Python, **CinéSearch** offre des capacités de recherche avancées sur un dataset de près de 5 000 films, avec des fonctionnalités allant de la simple recherche par titre aux requêtes multi-critères dynamiques, en passant par la tolérance aux fautes de frappe et l'auto-complétion.

La conteneurisation via Docker Compose a garanti un environnement reproductible et portable, tandis que l'indexation bulk a permis une ingestion performante des données. L'exploitation du Query DSL a mis en lumière la puissance d'Elasticsearch pour construire des requêtes complexes et pertinentes, et les agrégations ont permis d'extraire des statistiques riches sur le catalogue de films.

Ce projet a été l'occasion de maîtriser l'ensemble de la chaîne : déploiement d'infrastructure, modélisation d'un mapping, indexation de données réelles, écriture de requêtes variées, visualisation dans Kibana et développement d'une interface CLI ergonomique. Les compétences acquises — Docker, Elasticsearch, Python et Query DSL — constituent une base solide pour tout projet nécessitant un moteur de recherche performant à grande échelle.

7. Annexe — Captures d'Écran de l'Interface CinéSearch

Figure 2 — Résultat de la recherche par titre : Après une recherche sur “Inception”, l'application retourne 1 résultat affiché sous forme de card : titre en doré, année entre parenthèses, note IMDb, réalisateur et score de pertinence calculé par Elasticsearch.

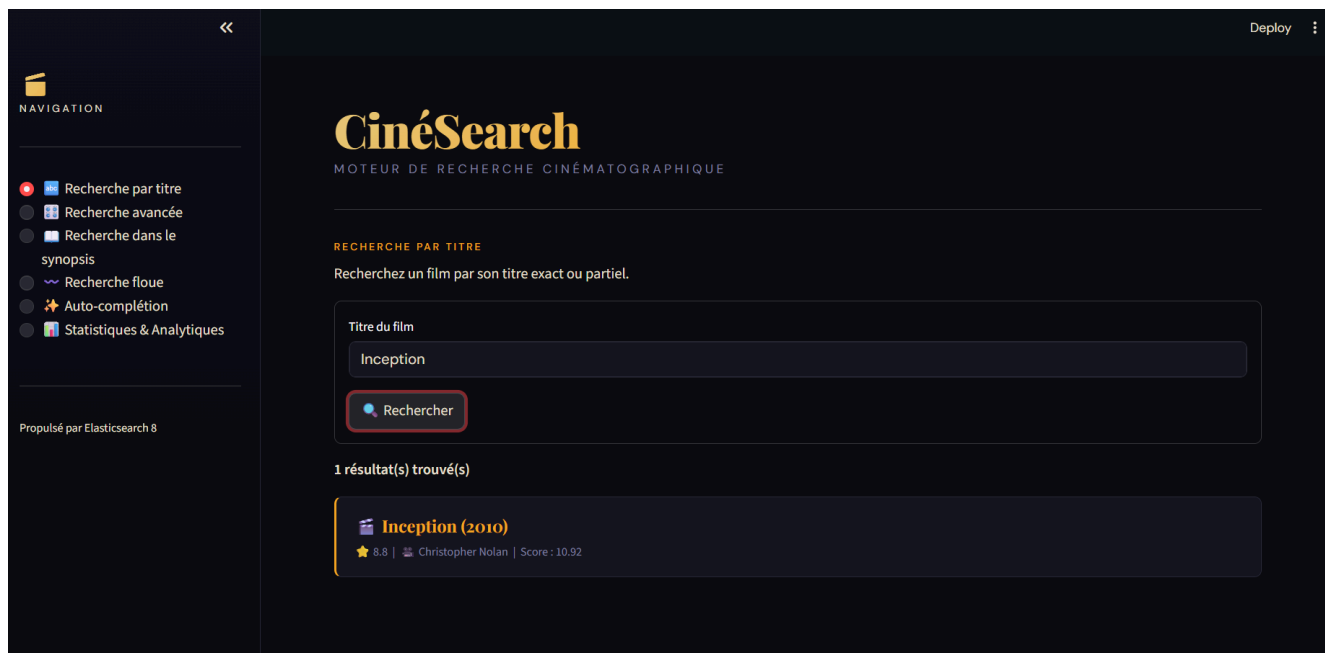


Figure 3 — Recherche avancée : La recherche avancée permet de combiner plusieurs critères simultanément. Ici, une recherche sur l'acteur “Bale” avec le réalisateur “Nolan” et une note minimum de 7.0 retourne 4 résultats : The Dark Knight Rises, The Dark Knight, The Prestige et Batman Begins.

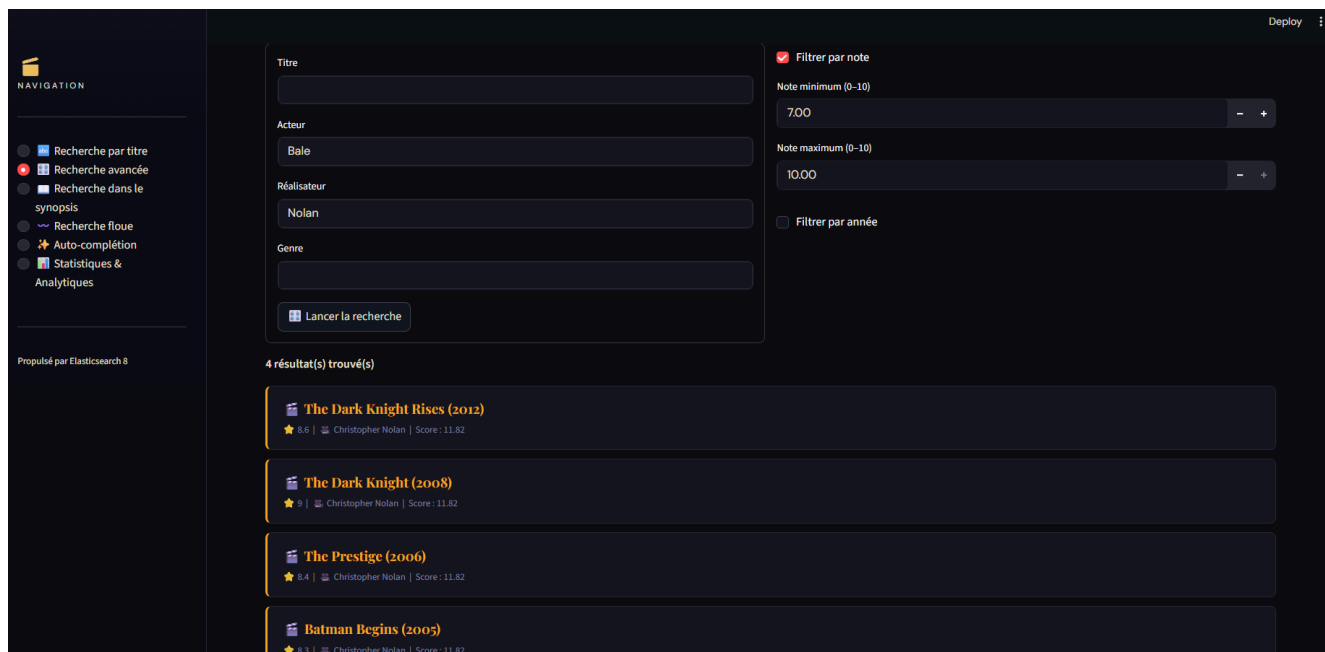


Figure 4 — Recherche dans le synopsis : La recherche dans le synopsis sur le mot-clé “war” retourne 170 résultats. Les fragments de synopsis sont affichés sous chaque film avec le terme recherché mis en évidence en gras.

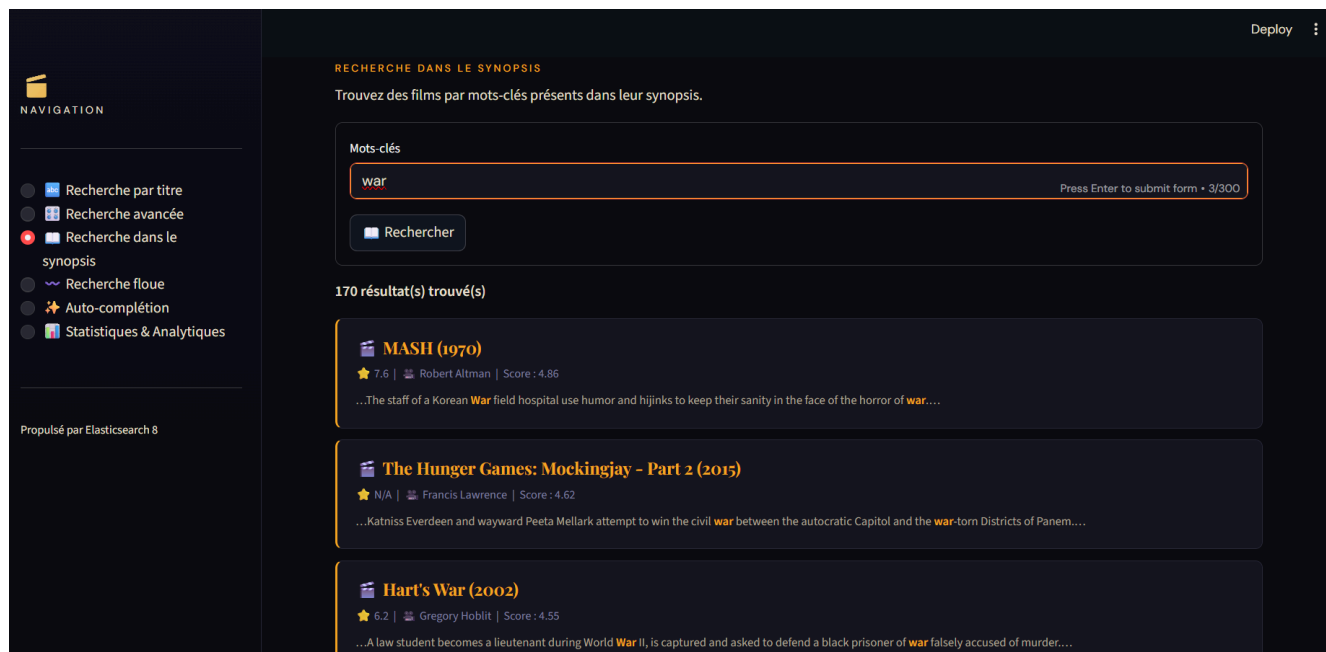


Figure 5 — Recherche floue : La recherche floue avec “Inception” (fuzziness = 2 — Modérée) retourne 2 résultats dont Inception, démontrant la tolérance aux fautes de frappe.

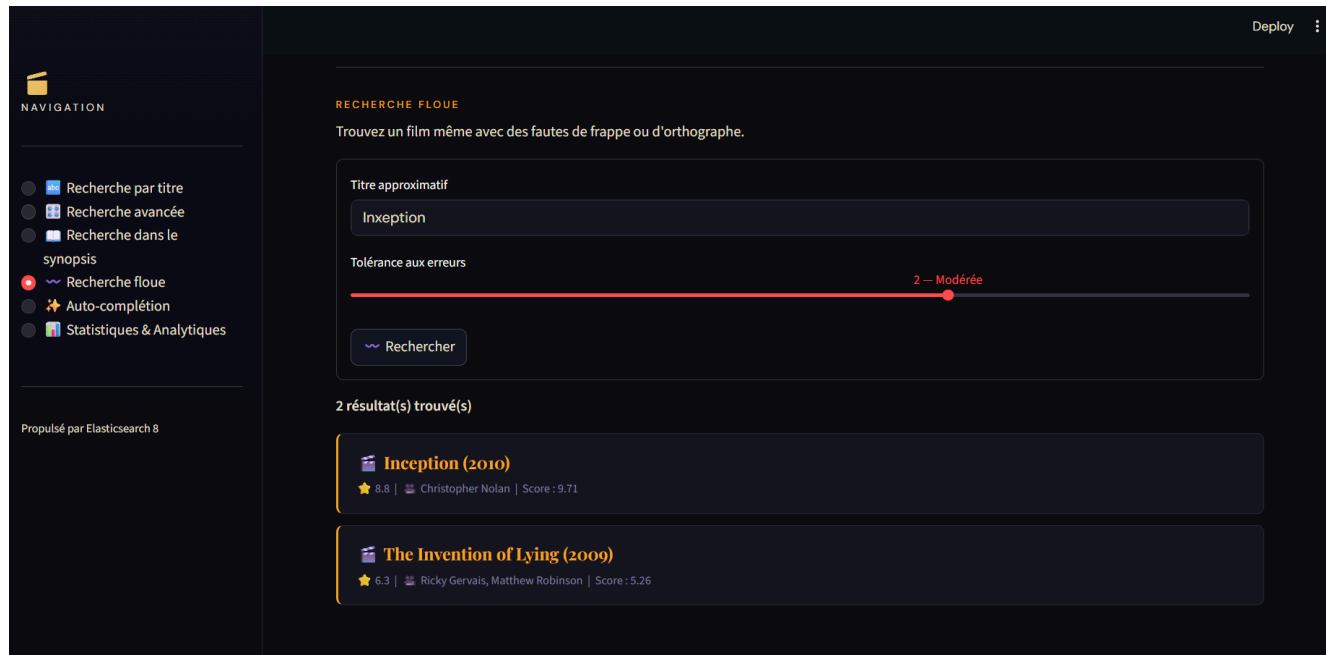


Figure 6 — Auto-complétion : La saisie du préfixe “bat” suggère 18 titres commençant par ces lettres : Battleship, Batoru rowaiaru, Battlefield Earth, Batman...

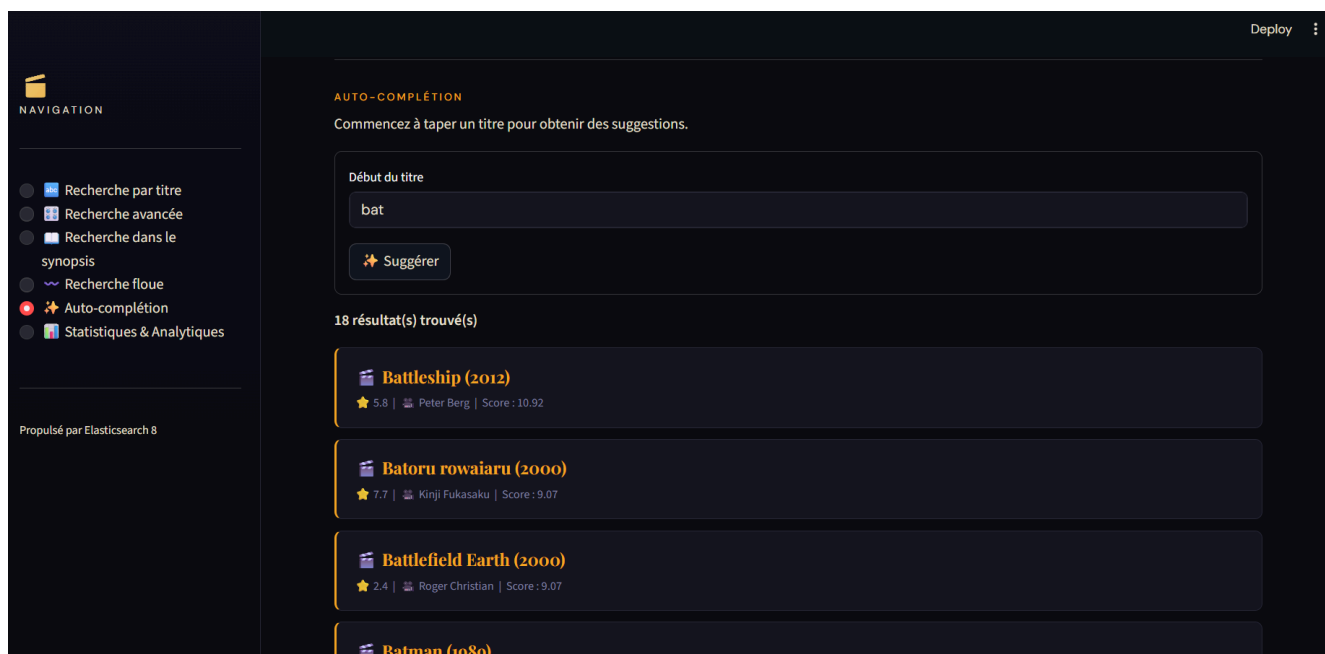
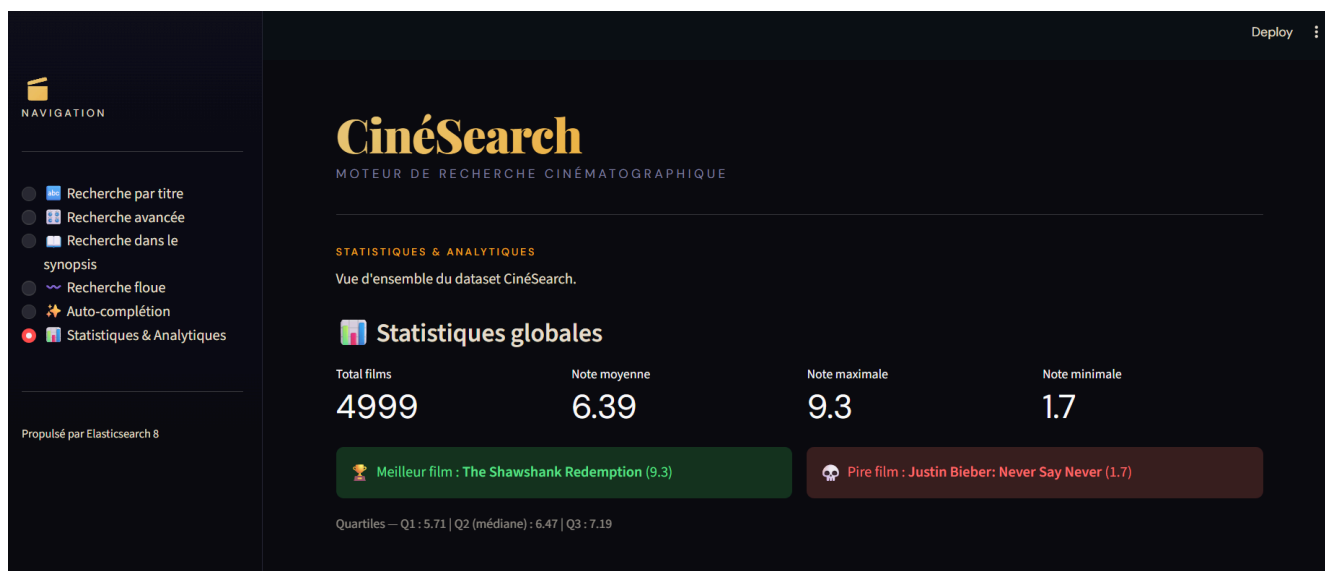


Figure 7 — Statistiques globales et tops : La page Statistiques & Analytics affiche les métriques globales (4999 films, note moyenne 6.39, meilleur film The Shawshank Redemption à 9.3, pire film Justin Bieber: Never Say Never à 1.7), ainsi que les top 10 genres, réalisateurs et acteurs sous forme de tableaux côte à côte.



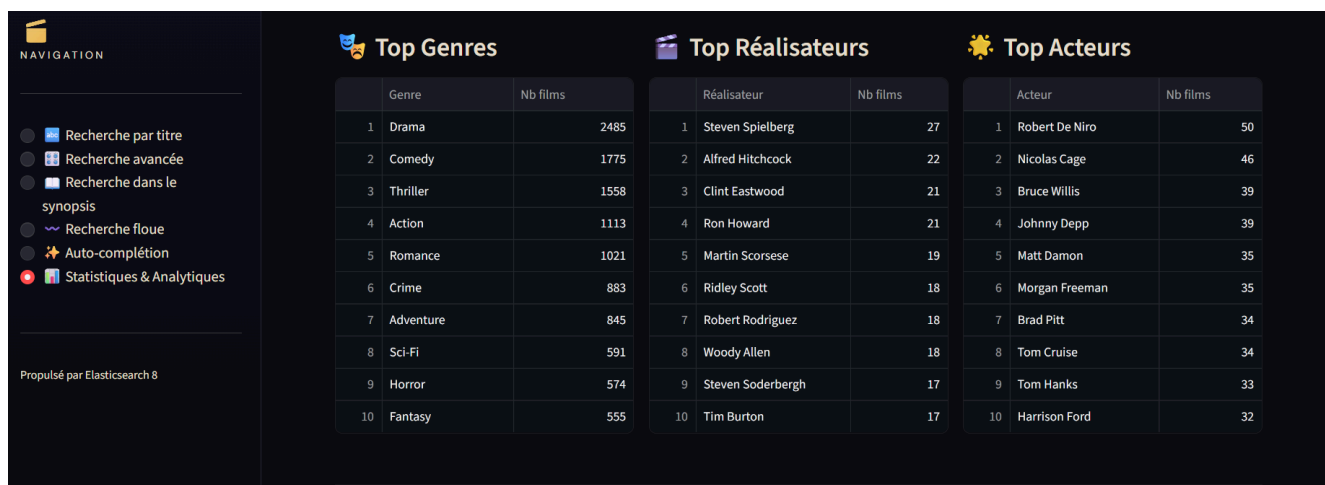


Figure 8 — Distribution par décennie : Un graphique en barres horizontales montre la distribution des films par décennie. Les années 2000s et 2010s dominent largement avec respectivement ~1 740 et ~1 635 films.

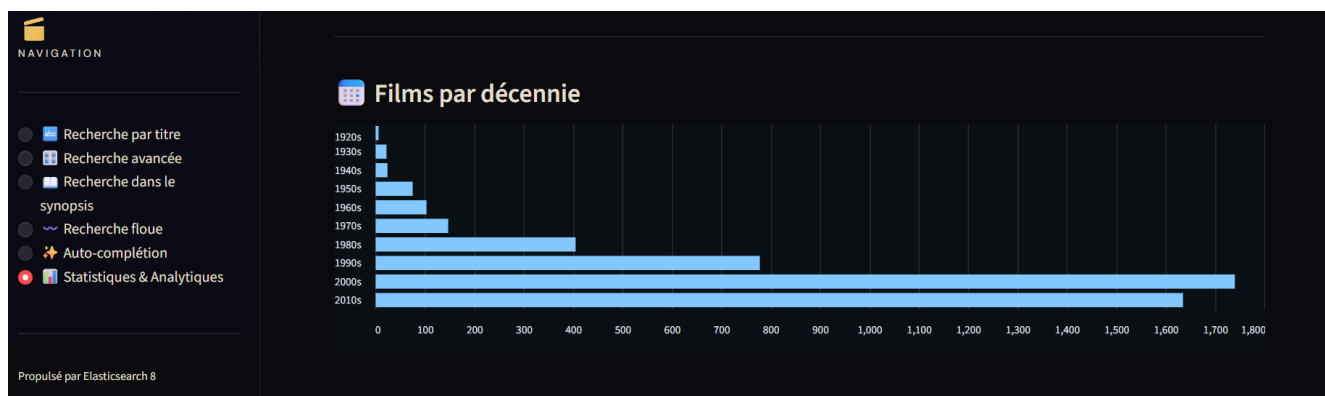


Figure 9 — Note moyenne par année : Un graphique linéaire illustre l'évolution de la note moyenne par année depuis 1925. On observe une légère tendance à la baisse des notes moyennes sur les dernières décennies, probablement due à l'augmentation du volume de films indexés.

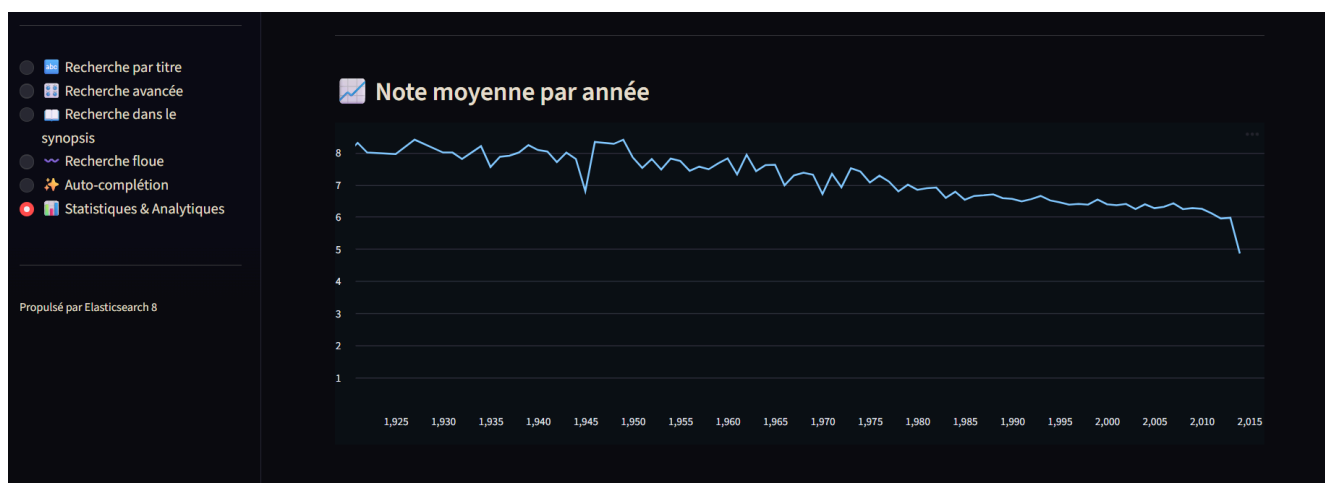


Figure 10 — Genres et réalisateurs les mieux notés : Deux tableaux côte à côte affichent les genres les mieux notés (Film-Noir en tête avec 8.16 de moyenne) et les réalisateurs les mieux notés avec un minimum de 3 films (Christopher Nolan en tête avec 8.29).

